

Contents

1	Pointer Basics	3
1.1	What is a pointer?	3
1.2	Pointer variables	3
1.3	Indirection	3
2	Addresses and indirection	4
2.1	What is the relationship between a pointer and an array (or string)?	4
2.1.1	C strings	4
2.1.2	Continuation	4
2.2	Arrays, addresses, and pointers	5
2.3	Multiple indirection	6
2.4	Multiple indirection with integers	6
2.5	Multiple indirection with strings	8
2.6	Indirection and commandline args	9
2.7	Generic pointers	10
2.7.1	Example	10
2.8	Allocating memory	11
2.8.1	Example	12
2.8.2	Conclutions	12
2.9	Malloc and sizeof	12
2.10	Functions that cause errors or warnings	13
2.11	calloc	13
2.11.1	Parameter differences between calloc and malloc	14
2.11.2	Null pointers	15
2.12	free	15
2.13	realloc	15
2.14	Pointer arithmetic	16
2.15	Calculating an array index	17
2.16	Pointers to structs	19
2.17	Data type alignment	19
2.18	Type alignment on boundaries	20
2.19	Type alignment and pointer arithmetic	20

Chapter 1

Pointer Basics

1.1 What is a pointer?

- Some languages don't let you use pointers.
- In C there is no getting away from pointers.
- C is a low level language and this is why it let's you use pointers.
- So what is a pointer? A pointer is a variable that points to an address in memory.
- What is an address? Think of the computer memory as a large series of locations where data is stored, all of these locations are labeled. A pointer is able to reference an address.

1.2 Pointer variables

- They are declared using the asterisk between the data type and the variable name.

```
#include <stdio.h>

int main(int argc, char **argv)
{
    int num2;
    int *numPtr;
    int num2;
    num = 100;
    numPtr = &num;
    num2 = *numPtr;
    printf("num=%d, numPtr=%d, address of num=%d, num2=%d\n", num, numPtr, &num, num2);
    return 0;
}
```

Figure 1.1: C program

1.3 Indirection

- If you have a pointer variable that hold the address in memory at which some other piece of data is stored, the address by itself is not really useful, but we can get the value in memory which is stored at the pointed location by using indirection or dereferencing.
- Given a pointer variable you can access the data being pointed to by adding an asterisk to the variable.

```
num2 = *numPtr;
```

Figure 1.2: Indirection operation

Chapter 2

Addresses and indirection

2.1 What is the relationship between a pointer and an array (or string)?

- Arrays in C are sequential data items stored in some direction in memory.
- The address of the array, is the same as the address of the first item in the array.
- The name of the array is also the address of the array.
- Let's see how strings are used in C.

2.1.1 C strings

- In C strings are just arrays of characters.
- C treats the null character `'\0'` as the string's terminator.

```
1 #include <stdio.h>
2 int main() {
3     char str1[] = "Hello world";
4     printf("%s %c %d %d\n", str1, str1[0], &str1, &str1[0], str1);
5     return 0;
6 }
7 /* Output:
8 Hello world H 312473588 312473588
9 */
```

- Note that the string is stored in location 312473588 in RAM. When printing the address of the first element of the array and the address of the array we get exactly the same location.

2.1.2 Continuation

- The address of an array is the same as the address of the first element in the array because it is where the array begins. The name of the array is mapped to the address of the array by the compiler, thus we can say that the name of the array is the location of the array and the location of the array is the location of the first member of the array.
- Array is a location in memory, thus an array is a pointer. However all thought an array is a pointer, it is not handled the same as a pointer in C. A pointer variable needs to be handled differently than an array, although behind the scenes an array is a pointer.

2.2 Arrays, addresses, and pointers

- Notice the following code and it's output:

```
1 #include <stdio.h>
2 int main() {
3     char str1[] = "Hello"; // Array
4     char *str2 = "Goodbye"; // Pointer
5     printf("%u %u %s\n", &str1, str1, str1);
6     printf("%u %u %s\n", &str2, str2, str2);
7     return 0;
8 }
9 /*
10 Output:
11 3558866218 3558866218 Hello
12 3558866208 686526464 Goodbye
13 */
```

- Notice that when outputting the array str1's address and the array itself we get the same one, this is because the array is the pointer to the string "Hello". Notice that when outputting the array str2's address we get a different address than the str2 variable, that is because when we get the address of a pointer, a pointer has a location in memory, and the value of a pointer references the string, the pointer has a location in memory, different to the array pointer, the pointer is not the array in this case, the pointer is a separate variable that makes reference to the array as it's value.
- Thus the second direction is different because we are printing the pointer address and the array address, which is not the same.
- An array variable such as str1 is an address, it is it's own address, the array variable is the address of the array.
- But a pointer variable is not an array, it is a variable that stores the address of the array. The pointer variable itself is stored in one address, and the array is stored in another.
- Think of the example we gave earlier, the giant lot of warehouses that are labeled and we need to go to warehouse 13.



- The card is like a pointer variable, it is not the location, it contains the value of the location of the warehouse. The warehouse is like the array, it is not just a reference to the location, it is the location. The pointer is like the card containing the text "warehouse 13" the card is not the location, it contains the location that we need to go to, the card is not the warehouse 13.

Data	'G'	'o'	'o'	'd'	'b'	'y'	'e'	'\0'
Address	4206628							

Name	str2
Data	4206628
Address	2696740

- Let's try out another example:

```

1 #include <stdio.h>
2 int main() {
3     char str1[] = "Hello"; // Array
4     char *str2 = str1; // Pointer
5     printf("%u %u %s\n", &str1, str1, str1);
6     printf("%u %u %s\n", &str2, str2, str2);
7     return 0;
8 }
9 /*
10 Output:
11 3911186266 3911186266 Hello
12 3911186256 3911186266 Hello
13 */

```

- Notice that get the different numbers in the second printing because now the pointer variable is pointing to the array. See how each of them are different? One is the reference to the location of the array and the other is the array.

2.3 Multiple indirection

- A pointer can also point to another pointer.
- For example: `int **ppi;`
- When we use a pointer to access data at an address we call it indirection or dereferencing.
- Using a pointer to access data from another pointer is called multiple indirection. Because it requires more than one step.
- Why would we ever need multiple indirection?
 - We might have a list of lists, that are accessed using pointer to pointer.

2.4 Multiple indirection with integers

```

1 #include <stdio.h>
2 #define LENGTH 3
3 int data[LENGTH]; // array of integers.
4 int main(int argc, char **argv) {
5     int *pi; // pointer to an int.
6     int **ppi; // pointer to pointer to int.
7     printf("multiple indirection example\n");
8     // initialize our integer array.
9     for (int i = 0; i < LENGTH; i++) {
10         data[i] = i;
11     }

```

```

12     for (int i = 0; i < LENGTH; i++) {
13         printf("%d\n", data[i]);
14     }
15     /*
16     A: simple pointer to a pointer to an integer.
17     */
18     pi = data; // the address of data is assigned to pi.
19     ppi = &pi; // the ppi points to the pointer pi.
20     for (int i = 0; i < LENGTH; i++) {
21         printf("\nLoop[%d] array address is %u\n", i, data);
22         printf("item pointed to by pi is %d\n", *pi);
23         printf("item pointed to by ppi is %u\n", *ppi);
24         printf("item pointed to by double indirection of ppi is %d\n\n", **ppi);
25         printf("The address of pi is %u\n\tand the value of pi (what it points to) is\n\t\t↪ %u\n\n", &pi, ppi);
26         pi += 1;
27     }
28     return 0;
29 }
30 /*
31 multiple indirection example
32 0
33 1
34 2
35
36 Loop[0] array address is 341364800
37 item pointed to by pi is 0
38 item pointed to by ppi is 341364800
39 item pointed to by double indirection of ppi is 0
40
41 The address of pi is 727710864
42     and the value of pi (what it points to) is 727710864
43
44
45 Loop[1] array address is 341364800
46 item pointed to by pi is 1
47 item pointed to by ppi is 341364804
48 item pointed to by double indirection of ppi is 1
49
50 The address of pi is 727710864
51     and the value of pi (what it points to) is 727710864
52
53
54 Loop[2] array address is 341364800
55 item pointed to by pi is 2
56 item pointed to by ppi is 341364808
57 item pointed to by double indirection of ppi is 2
58
59 The address of pi is 727710864
60     and the value of pi (what it points to) is 727710864
61 */

```

- Notice the above code.

- The area we are really interested in understanding is the for loop with the multiple indirection. Lets have a closer look:

```
1 for (int i = 0; i < LENGTH; i++) {
```

- Iterating from i=0 to the length of the array.

```
1 printf("\nLoop[%d] array address is %u\n", i, data);
```

- This statement will print the index which we are using right now. And the address of the array data.

```
1 printf("item pointed to by pi is %d\n", *pi);
```

- This statement is used to print the value of the pointer pi.

```
1 printf("item pointed to by ppi is %u\n", *ppi);
```

- This statement is used to print the value of ppi, which is another pointer, thus the value being printed is the decimal form location of the other pointer in memory.

```
1 printf("item pointed to by double indirection of ppi is %d\n\n", **ppi);
```

- This statement is using double indirection to access the data pointed to by the double pointer.



- Notice the ****ppi**, the double asterisk is to dereference twice. Get the value of the value of the pointer.

```
1 printf("The address of pi is %u\n\tand the value of pi (what it points to) is  
↪ %u\n\n", &pi, ppi);
```

- This statement is used to print the address of the pointer pi and also to print the pointer ppi, ppi is pointing to the address of pi, thus the address will always be the same in this program.

```
1 pi += 1;  
2 }
```

- pi is a pointer and it will be incremented by 1, since it is an int pointer it will be incremented by the sizeof one int. This is called pointer arithmetic.

2.5 Multiple indirection with strings

```
1 #include <stdio.h>  
2 #define LENGTH 3  
3 char *words[LENGTH]; // array of char.  
4 int main(int argc, char **argv) {  
5     char *pc; // pointer to an char.
```

```

6  char **ppc; // pointer to pointer to char.
7  printf("multiple indirection example\n");
8  // initialize our integer array.
9  words[0] = "zero";
10 words[1] = "one";
11 words[2] = "two";
12 for (int i = 0; i < LENGTH; i++) {
13     printf("%s\n", words[i]);
14 }
15 /*
16 B: simple pointer to an array of strings.
17    The same as a pointer to a pointer to a character.
18 */
19 printf("\nNow print the chars of each string...\n");
20 ppc = words; // ppc is pointing to the array location of words.
21 for (int i = 0; i < LENGTH; i++) {
22     ppc = words + i; // initialize the pointer to a pointer to a character
23     pc = *ppc; // loop over each string.
24     while (*pc != 0) {
25         printf("%c ", *pc); // process each character in a string.
26         pc += 1; // print out each character of the string individual.
27     }
28     printf("\n");
29 }
30 return 0;
31 }
32 /* Output:
33 multiple indirection example
34 zero
35 one
36 two
37
38 Now print the chars of each string...
39 z e r o
40 o n e
41 t w o
42 */

```

2.6 Indirection and commandline args

- As you know C programs accept command line arguments, we send these to the main function using the arguments argc and argv. The argv is a pointer to a pointer to a char.

```

1  int main(int argc, char **argv) {
2      ...
3  }

```

- The argc is the amount of arguments passed to the program, the argv variable is a pointer to a pointer of chars, which is an array of strings.
- Let's see an example:

```

1  #include <stdio.h>
2  int main(int argc, char **argv) {

```

```

3     for (int i = 0; i < argc; i++) {
4         printf("arg %d is %s\n", i, argv[i]);
5     }
6     printf("\n\n");
7     for (int i = 0; i < argc; i++) {
8         printf("arg %d is %s\n", i, *argv);
9         argv++;
10    }
11    return (0);
12 }
13 /* Output:
14 arg 0 is a
15 arg 1 is hello
16 arg 2 is this
17 arg 3 is an
18 arg 4 is argv
19 arg 5 is argument
20 arg 6 is setence
21
22
23 arg 0 is a
24 arg 1 is hello
25 arg 2 is this
26 arg 3 is an
27 arg 4 is argv
28 arg 5 is argument
29 arg 6 is setence
30 */

```

2.7 Generic pointers

- Sometimes we have a function that needs to perform an operation using several inputs. If the inputs are double, float, int or long does not matter, this case you would be inclined to write the same function for all the data types, but you would not be advised to not duplicate code. We can write just one funtion that takes in a generic pointer instead.
- A generic pointer is a pointer to void. To use it you must cast it into a data type.
- This is useful since it gives us more liberty and allows us to not repeat code.

2.7.1 Example

```

1 #include <stdio.h>
2
3 #define LENGTH 3
4 int data[LENGTH];
5 char *words[LENGTH];
6
7 int main(int argc, char **argv) {
8     void *gp;
9     printf("generic pointer example\n");
10
11     // initialize out integer array.

```

```

12     for (int i = 0; i < LENGTH; i++) {
13         data[i] = i;
14     }
15     for (int i = 0; i < LENGTH; i++) {
16         printf("%d\n", data[i]);
17     }
18     // initialize our string array.
19     words[0] = "zero";
20     words[1] = "one";
21     words[2] = "two";
22     for (int i = 0; i < LENGTH; i++) {
23         printf("%s\n", words[i]);
24     }
25     // example of a generic pointer.
26     gp = data;
27     printf("\ndata array address is %p\n", gp);
28     // cast to int *. then dereference.
29     printf("item pointed to by gp is %d\n", *(int*)gp);
30     gp = (int*)gp + 1; // casting to a pointer to int. add 1.
31     printf("item pointer to by gp is now %d\n", *(int*)gp);
32     // now using the pointer to point to a string.
33     gp = words;
34     printf("\nwords arra address is %p", gp);
35     // cast to a char ** and then dereference the pointer.
36     printf("item pointed to by gp is %s\n", *(char**)gp);
37     gp = (char**)gp + 1; // dereference and now add one
38     printf("item pointed to by gp is now %s", *(char**)gp);
39     return (0);
40 }
41 /* Output:
42 0
43 1
44 2
45 zero
46 one
47 two
48
49 data array address is 00007ff695f6d040
50 item pointed to by gp is 0
51 item pointer to by gp is now 1
52
53 words arra address is 00007ff695f6d050item pointed to by gp is zero
54 item pointed to by gp is now one
55 */

```

2.8 Allocating memory

- When we created arrays in the previous examples they were created we assigned a string. The memory is allocated for these bytes of data. But sometimes we don't know how much memory to allocate at compile time, we need the ability to allocate memory at run time.
- We can handle memory allocation using the malloc function.

- The malloc function returns a pointer to void, it allocates the memory we need at run time, thus we can request and free memory at run time as needed.
- The malloc function allocates memory in an area called the heap, normal arrays are allocated on the stack.

2.8.1 Example

```

1 #include <stdio.h>
2 #include <string.h>
3 #include <malloc.h>
4
5 #define MAXSTRLEN 100
6
7 char *string_function(char *astring) {
8     char *s;
9     s = (char*)malloc(MAXSTRLEN);
10    s[0] = 0; // initialize first char to null.
11    strcat(s, "Hello ");
12    strcat(s, astring);
13    strcat(s, "\n");
14    return s;
15 }
16
17 int main(int argc, char **argv){
18     printf(string_function("Fred"));
19     printf(string_function("Gussie Fink-Nottle"));
20     return 0;
21 }
22
23 /* Output:
24 Hello Fred
25 Hello Gussie Fink-Nottle
26 */

```

2.8.2 Conclutions

- The implementation above has some problem, namely that there could be a string passed in that is larger than 100 bytes, and therefore we can have a buffer overflow problem. Also all space allocated in the heap needs to be explicitly freed, this is a concept we will cover later.
- Thus the above implementation is flawed.

2.9 Malloc and sizeof

- Let's see how we can allocate just the right amount of memory:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     char *s;
7     int stringsize;

```

```

8     stringsize = sizeof("hello"); // sizeof operator calculates the size of the
    ↪ data.
9     printf("size of 'hello' is %d\n", stringsize);
10
11     s = (char*)malloc(stringsize);
12     // if malloc returns null then there was an error and we must terminate the
    ↪ program.
13     if (s == NULL) {
14         printf("malloc failed.");
15         exit(0);
16     }
17     // now copy the string into the newly allocated string.
18     strncpy(s, "hello", stringsize);
19
20     // and change the first character (just to show we can)
21     printf("s is %s\n", s);
22     s[0] = 'c';
23     printf("s is now %s\n", s);
24     return (0);
25 }
26 /* Output:
27 size of 'hello' is 6
28 s is hello
29 s is now cello
30 */

```

2.10 Functions that cause errors or warnings

- Sometimes when using some functions, a warning will appear that will tell you that the function you are using is deprecated. There are sometimes reasons to still use them for example, to make the functions compatible with any C compiler.
- In many cases the best option will be to use the recommended function. Or you could define the constant `_CRT_SECURE_NO_WARNINGS` to turn off the deprecation warnings.
- By placing the constant:

```

1 #define _CRT_SECURE_NO_WARNINGS

```

- The compiler should stop pestering you about the deprecation status of the functions you use.

2.11 calloc

- There is a function called `calloc` that is similar to `malloc`. The `calloc` function performs everything the `malloc` function would but it also performs the additional step of clearing the memory with nulls.
- Uninitialized memory contains junk. You could allocate memory and the memory you allocate can contain anything, thus it is prone to errors and bugs.
- If you allocate memory with `calloc` the memory is allocated and cleared out with all 0s.
- If you are careful and always initialize all the memory after using `malloc`, then by all means use `malloc`. Otherwise use `calloc`.

2.11.1 Parameter differences between calloc and malloc

- malloc just takes in the amount of memory in bytes to be allocated.

```
1 s = (char*)malloc(6);
```

- calloc takes in two parameters, first the amount of elements to be allocated, then the size of each element.

```
1 s = (char*)calloc(6, sizeof(char)); // total space allocated 6*1.
```

- Basically the calloc function will multiply the amount of elements with the space required by each element.
- Notice the following example, we allocate memory and print the data contained within that space. Look that random numbers are thrown. Then using calloc, it is always 0s.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     char *s;
7     int *p;
8     s = (char*)malloc(6);
9     for (int i = 0; i < 6; i++) { printf("s[%d]=%d\n", i, s[i]); }
10    printf("\n");
11
12    s = (char*)calloc(6, sizeof(char));
13    for (int i = 0; i < 6; i++) { printf("s[%d]=%d\n", i, s[i]); }
14    free(s);
15    printf("\n");
16
17    p = (int*)calloc(6, sizeof(int));
18    for (int i = 0; i < 6; i++) { printf("p[%d]=%d\n", i, p[i]); }
19    free(p);
20    return (0);
21 }
22 /* Output:
23 s[0]=16
24 s[1]=104
25 s[2]=119
26 s[3]=12
27 s[4]=-108
28 s[5]=1
29
30 s[0]=0
31 s[1]=0
32 s[2]=0
33 s[3]=0
34 s[4]=0
35 s[5]=0
36
37 p[0]=0
38 p[1]=0
```

```

39 p[2]=0
40 p[3]=0
41 p[4]=0
42 p[5]=0
43 */

```

2.11.2 Null pointers

- Both calloc and malloc will return NULL pointer if the allocation fails. A null pointer is a pointer whose value is 0. By convention, a pointer with the value 0 is a pointer that does not contain a valid address.
- We can test for a null pointer by comparing it to the NULL constant.

2.12 free

- Once memory is allocated it is no longer available to the rest of your program.
- So you can run out of memory if you don't free it.
- This only applies to memory inside the heap, the stack memory is managed by the compiler.
- Keeping memory allocated is waste.
- Memory allocated but not used is called a memory leak. This can be a real problem for devices that don't have much memory, such as embedded devices or even device controllers.
- When the memory is no longer need you must free it.
- To free allocated memory you just need to call free with the pointer you used to allocate.
- In the previous example we used free:

```

1 free(s); // the only parameter is the pointer to the allocated data.

```

- If you call free on a variable that was never allocated or try to use allocated memory after you call free then your program can go horribly wrong.

2.13 realloc

- To change the size of allocated memory you might use the realloc function.
- Suppose we've allocated enough space for the characters 'hello'. And now we require enough space for 'hello world', if we were to simply strcpy this new string into a buffer with just enough space to say 'hello' we will have a buffer overflow. And the resulting program will be unpredictable because we are accessing unreserved memory space.

```

1 char *s;
2 int i;
3
4 i = sizeof("hello");
5 s = (char*)malloc(i);
6 strncpy(s, "hello world"); // buffer overflow
7 printf("s is %s\n", s);
8

```

```

9 strcat(s, " world"); // program crashes, buffer overflow.
10 printf("s is now %s\n", s);

```

- Using realloc we can expand or shrink the size of allocated memory. The solution is the following:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int main() {
6     char *s;
7     int i;
8
9     i = sizeof("hello");
10    s = (char*)malloc(i);
11    strncpy(s, "hello");
12    printf("s is %s\n", s);
13
14    realloc(s, 12);
15    strcat(s, " world"); // program crashes, buffer overflow.
16    printf("s is now %s\n", s);
17
18    free(s);
19    return (0);
20 }
21 /* Output:
22 s is hello
23 s is now hello world
24 */

```

2.14 Pointer arithmetic

- When iterating arrays, we've advanced to the next element of the array using the `ptr += 1` statement, but the elements are variable in size, one element can be an int which occupies 4 bytes, adding 1 would not advance to the next element in this case. This is because despite adding 1 conceptually doesn't land us in the next element. The compiler behind the scenes adds 1 times the `sizeof` of the element to get the element.
- The compiler does this with pointers because the what is being pointed to only makes sense to advance it by the size of the elements of the array not by individual bytes.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define COUNT 4
6
7 int main() {
8     int *p;
9     int a[COUNT];
10
11    for (int i = 0; i < COUNT; i++) {
12        a[i] = i;

```

```

13     }
14     p = a;
15     printf("Address of a is %p; value of p is %p [%d]; value pointed to by p is
    ↪ %d\n", a, p, p, *p);
16
17     p = p + 1;
18     printf("Address of a is %p; value of p is %p [%d]; value pointed to by p is
    ↪ %d\n", a, p, p, *p);
19
20     p = p + 2;
21     printf("Address of a is %p; value of p is %p [%d]; value pointed to by p is
    ↪ %d\n", a, p, p, *p);
22
23     p = p + 1; // exceded the bounds of the array. Run time overflow.
24     printf("Address of a is %p; value of p is %p [%d]; value pointed to by p is
    ↪ %d\n", a, p, p, *p);
25
26     return (0);
27 }
28 /* Output:
29 Address of a is 00000055a25ffa10; value of p is 00000055a25ffa10 [-1570768368];
    ↪ value pointed to by p is 0
30 Address of a is 00000055a25ffa10; value of p is 00000055a25ffa14 [-1570768364];
    ↪ value pointed to by p is 1
31 Address of a is 00000055a25ffa10; value of p is 00000055a25ffa1c [-1570768356];
    ↪ value pointed to by p is 3
32 Address of a is 00000055a25ffa10; value of p is 00000055a25ffa20 [-1570768352];
    ↪ value pointed to by p is -1570768352
33 */

```

2.15 Calculating an array index

- As you saw, adding one to a pointer increases its value by the size in bytes of the data its pointing to. If its an array of ints the increments will take place in steps of 4, if its an array of chars it will take place in steps of 1.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 #define COUNT 4
6
7 int main() {
8     int *pa;
9     float *pb;
10    double *pc;
11    long long int *pd;
12    short int *pe;
13    long int *pf;
14
15    int a[COUNT];
16    float b[COUNT];
17    double c[COUNT];

```

```

18     long long int d[COUNT];
19     short int e[COUNT];
20     long int f[COUNT];
21
22     printf("sizes are: int %d; float %d; double %d; long long int %d; short int
    ↪ %d; long int %d\n", sizeof(int),
23         sizeof(float), sizeof(double), sizeof(long long int), sizeof(short int),
    ↪ sizeof(long int));
24
25     pa = a; pb = b; pc = c; pd = d; pe = e; pf = f;
26     for (int i = 0; i < COUNT; i++) {
27         a[i] = b[i] = c[i] = d[i] = e[i] = f[i] = i;
28     }
29
30     pa += 1;
31     pb += 1;
32     pc += 1;
33     pd += 1;
34     pe += 1;
35     pf += 1;
36
37     printf("address of a is %p [%u]; value of pa is %p [%u]; value pointed to by
    ↪ pa is %d\n", a, a, pa, pa, *pa);
38     printf("address of b is %p [%u]; value of pb is %p [%u]; value pointed to by
    ↪ pb is %f\n", b, b, pb, pb, *pb);
39     printf("address of c is %p [%u]; value of pc is %p [%u]; value pointed to by
    ↪ pc is %f\n", c, c, pc, pc, *pc);
40     printf("address of d is %p [%u]; value of pd is %p [%u]; value pointed to by
    ↪ pd is %lld\n", d, d, pd, pd, *pd);
41     printf("address of e is %p [%u]; value of pe is %p [%u]; value pointed to by
    ↪ pe is %hd\n", e, e, pe, pe, *pe);
42     printf("address of f is %p [%u]; value of pf is %p [%u]; value pointed to by
    ↪ pf is %ld\n", f, f, pf, pf, *pf);
43
44     return 0;
45 }
46 /*
47 Output:
48 sizes are: int 4; float 4; double 8; long long int 8; short int 2; long int 4
49 address of a is 000000f5afdff9a0 [2950691232]; value of pa is 000000f5afdff9a4
    ↪ [2950691236]; value pointed to by pa is 1
50 address of b is 000000f5afdff990 [2950691216]; value of pb is 000000f5afdff994
    ↪ [2950691220]; value pointed to by pb is 1.000000
51 address of c is 000000f5afdff970 [2950691184]; value of pc is 000000f5afdff978
    ↪ [2950691192]; value pointed to by pc is 1.000000
52 address of d is 000000f5afdff950 [2950691152]; value of pd is 000000f5afdff958
    ↪ [2950691160]; value pointed to by pd is 1
53 address of e is 000000f5afdff948 [2950691144]; value of pe is 000000f5afdff94a
    ↪ [2950691146]; value pointed to by pe is 1
54 address of f is 000000f5afdff930 [2950691120]; value of pf is 000000f5afdff934
    ↪ [2950691124]; value pointed to by pf is 1
55 */

```

- Notice that advancing the pointer by one is not always advancing it by one byte but advancing it by

the size in bytes of each element in the arrays.

- Sometimes the data types are different sizes on different machines, for example on a PC the size of the long int is 4 and on a mac it is 8 bytes. This is why its better to have the `+= 1` notation since different systems can advance the pointers according to their specific sizes and we are not left with the problem of hardcoding the wrong data type size.

2.16 Pointers to structs

- Structs vary in size according to the order of their attributes.
- For example the following struct may take the size of the sum of its data types.

```
1 typedef struct {
2     int a; // 4 bytes
3     int c; // 4 bytes
4     double b; // 8 bytes
5     long long int d; // 8 bytes
6 }
7 // 24 bytes in total?
```

- It maybe the case that the compiler determines the size of the struct to be 24, however the same struct with changed order of attributes will not be equivalent in size to the above struct:

```
1 typedef struct {
2     int a; // 4 bytes
3     double b; // 8 bytes
4     int c; // 4 bytes
5     long long int d; // 8 bytes
6 }
7 // 24 bytes in total? Now its 32 bytes.
```

- Merely changing the order of attributes prompts the compiler to add padding space and the size may no longer be 24. So to be safe NEVER ASSUME THE SUM OF THE ATTRIBUTES OF A STRUCT IS ITS SIZE, use the sizeof operator instead. sizeof returns the correct value regardless of what the true space allocated ends up being.
- To understand why the C compiler does not use exactly the required space to allocate structs we must understand how the C compiler alligns the data in memory. This is the content of the next lecture.

2.17 Data type alignment

- Reordering the fields of a struct may lead to the compiler using more space than just the sum of the sizes of the attributes.
- The data types are being aligned in memory. To understand this we must imagine that struct memory is layed out int rows of 8 bytes. Like so:

A	A	A	A	\0	\0	\0	\0
B	B	B	B	B	B	B	B
C	C	C	C	\0	\0	\0	\0
D	D	D	D	D	D	D	D

- When we put an int first the 4 remaining bytes of the 8 byte row are used as padded to accomodate the next data types which is a double, this one will take the whole row. Then again an int with 4 bytes of padding and then finally the double. By placing the ints consecutively we are avoiding padding and instead of taking 32 bytes we take only 24 since we did not use padding:

A	A	A	A	C	C	C	C
B	B	B	B	B	B	B	B
D	D	D	D	D	D	D	D

2.18 Type alignment on boundaries

- Data types of a certain size are aligned at boundaries of that size.
- An int will be aligned at boundaries of 4 bytes, a double will be aligned at boundaries of 8 bytes. The take on knowing this is not to predict the size of the struct, but to understand that the size of a struct cannot be assumed.
- Even if you manage to guess correctly using alignment and predicting the size, different machines have different sizes for the data types you are using.
- In short, always always use sizeof because it is the only way to ensure the size of the struct in bytes is calculated correctly.
- The ways structs are stored in memory is unpredictable accross different architectures and systems.
- Even making an apparently trivial change such as rearranging the fields may alter the amount of memory it needs.

2.19 Type alignment and pointer arithmetic

- Never assume the size of data types. Especially when allocating memory. Instead of hardcoding the presupposed size, use the sizeof operator to calculate the amount of memory needed.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  typedef struct {
6      int a; // 4 bytes
7      int c; // 4 bytes
8      double b; // 8 bytes
9      long long int d; // 8 bytes
10 } MYSTRUCT;
11
12 #define COUNT 4
13
14 int main() {
15     MYSTRUCT *p, *q;
16     void *v;
17     printf("size of MYSTRUCT = %d\n", sizeof(MYSTRUCT));
18     p = (MYSTRUCT*)calloc(COUNT, sizeof(MYSTRUCT));
19
20     for (int i = 0; i < COUNT; i++) {
21         p[i].a = i;
22         p[i].b = 1000000000000.0 + i;

```

```

23     p[i].c = i * 20;
24     p[i].d = 4294967296 + i;
25 }
26 q = p;
27
28 printf("[0] values:\na is %d\nb is %f\ncis %d\nd is%lld\n", q->a, q->b, q->c,
    ↪ q->d);
29 printf("addresses:\na is %p\nb is %p\nc is %p\nd is %p\n", &q->a, &q->b,
    ↪ &q->c, &q->d);
30
31 q = p + 1;
32
33 printf("[1] values:\na is %d\nb is %f\ncis %d\nd is%lld\n", q->a, q->b, q->c,
    ↪ q->d);
34 printf("addresses:\na is %p\nb is %p\nc is %p\nd is %p\n", &q->a, &q->b,
    ↪ &q->c, &q->d);
35
36 v = p + 1;
37 printf("struct at index 1\n");
38 for (int i = 0; i < sizeof(MYSTRUCT) / sizeof(int); i++) {
39     printf("v[%d]=%d\n", i, ((int*)v)[i]);
40 }
41
42 return (0);
43 }
44 /*
45 Output:
46 size of MYSTRUCT = 24
47 [0] values:
48 a is 0
49 b is 1000000000000.000000
50 cis 0
51 d is4294967296
52 addresses:
53 a is 0000025f4dda69a0
54 b is 0000025f4dda69a8
55 c is 0000025f4dda69a4
56 d is 0000025f4dda69b0
57 [1] values:
58 a is 1
59 b is 10000000000001.000000
60 cis 20
61 d is4294967297
62 addresses:
63 a is 0000025f4dda69b8
64 b is 0000025f4dda69c0
65 c is 0000025f4dda69bc
66 d is 0000025f4dda69c8
67 struct at index 1
68 v[0]=1
69 v[1]=20
70 v[2]=-1577050112
71 v[3]=1114446484
72 v[4]=1

```

```

73 | v[5]=1
74 | */

```

- See the padding space and the locations of attributes in the structs. You can see the padding explicitly by checking the pointer addresses.

2.20 Debugging C Programs

- The debugger is a general solution to all problems with c code. It allows us to see code execution step by step and see where it fails in order to correct it.
- See the followign example:

```

1  #include <stdio.h>
2
3  int main(int argc, char **argv) {
4      int i;
5      for (int i = 0; i < argc; i++) {
6          printf("arg %d is %s\n", i, argv[i]);
7      }
8      print("\n\n");
9
10     for (i = 0; i < argc; i++) {
11         printf("arg %d is %s\n", i, *argv);
12         argv += 1;
13     }
14     return 0;
15 }

```

- In all the red dots the program execution will stop, this means that you can see the state of the program and possibly spot the problem. The red dot is called a breakpoint.

- When a breakpoint stops program execution you can see the local variables and their values in a panel such as this:

Locals		
Name	Value	Type
argc	4	int
argv	0x00000000366290 (0x00000000366290 "E:\\Btw\\Course\\1\\T\\Projects\\C\\1\\VC\\char**")	char**
i	0	int

- You also have a console in which you can type names of variables and it will print the value of the variable in that moment in execution.

```

immediate window
argv
0x00000000366290 {0x000000003662f9 "one"}
&argv
0x000000000022f728 {0x00000000366290 {0x000000003662f9 "one"}}

```

2.21 Debugging Multiple Indirection

- Multiple indirection is whenever you dereference a pointer and the dereferenced pointer is another pointer, thus you need to dereference it again to get the value, sometimes its a tripple pointer and you need to dereference 3 times. Multiple indirection is in general whenever you need to dereference something more than once.
- Given the following code:

```

1  #include <stdio.h>
2  struct {
3      int ***ippp, **ipp, *ip, i;
4  } p;

```

```

5
6 int main(int argc, char **argv) {
7     p.ipp = &p.ipp;
8     p.ipp = &p.ip;
9     p.ip = &p.i;
10    p.i = 0x55555555;
11    return 0;
12 }

```

- Using a memory detail window in visual studio we can see the individual cells in memory which is very handy for this example:

– See the following code:

```

int main(int ac, char**av) {
    p.ipp = &p.ipp;
    p.ipp = &p.ip;
    p.ip = &p.i;
    p.i = 0x55555555;

    return 0;
}

```

– We can see these values reflected here in memory:

0x00FB94E0	00fb94e4
0x00FB94E4	00fb94e8
0x00FB94E8	00fb94ec
0x00FB94EC	55555555
0x00FB94F0	00000000

- Notice that all the pointers point to the integer, the difference is that they are chained, one references the other like so:

0x00FB94E0	00fb94e4	
0x00FB94E4	00fb94e8	0x00FB94E0 16487648
0x00FB94E8	00fb94ec	
0x00FB94EC	55555555	
0x00FB94F0	00000000	

0x00FB94E0	00fb94e4	
0x00FB94E4	00fb94e8	
0x00FB94E8	00fb94ec	
0x00FB94EC	55555555	
0x00FB94F0	00000000	

0x00FB94E0	00fb94e4	
0x00FB94E4	00fb94e8	
0x00FB94E8	00fb94ec	
0x00FB94EC	55555555	
0x00FB94F0	00000000	

Chapter 3

Lists and data structures

3.1 Arrays and Lists

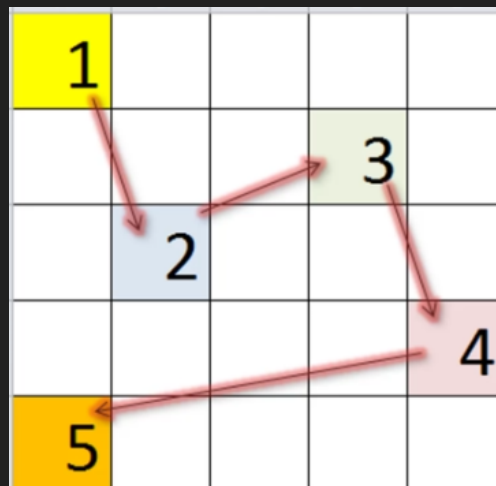
- As you know arrays have a fixed list, once it is declared of a certain size it stays that size unless you reallocate it to an array of a bigger size. But keep in mind that this is inefficient and the initial estimate might require more and more space.

3.2 What is a linked list?

- An array in memory is like an array of cells:



- With a linked list we do not allocate in a contiguous chunk but rather on an element by element bases.



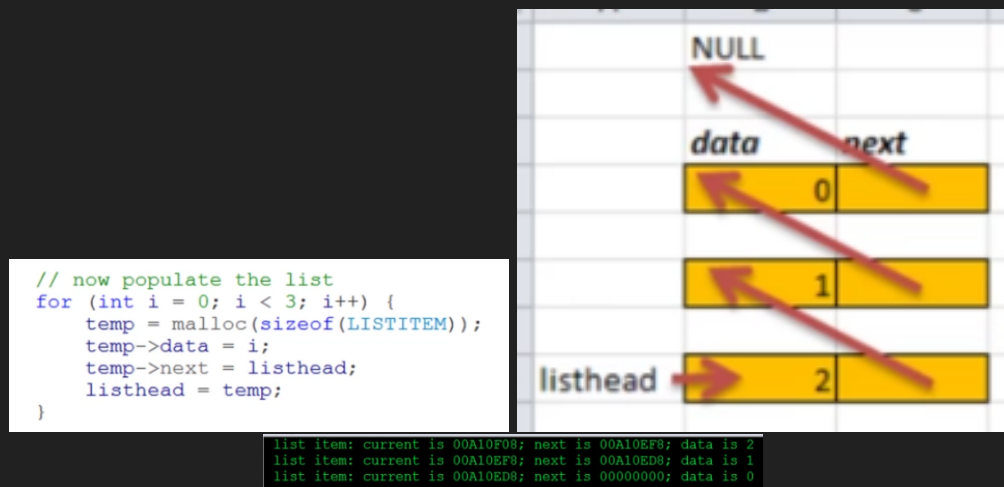
- What allows us to do this is pointers, because as I require an extra item, the location of the item is likely to be anywhere, not next to it. Thus I need to use pointers, and iterating a linked list uses pointers too. Each element of the list holds a pointer to the next element, if there is no next element then it will hold a null pointer.

3.3 Singly linked list

- Here is an example of a linked list of integers:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct listitem {
6     struct listitem *next;
7     int data;
8 } LISTITEM;
9
10 int main() {
11     LISTITEM *listhead, *temp;
12     listhead = NULL;
13
14     for (int i = 0; i < 3; i++) {
15         temp = malloc(sizeof(LISTITEM));
16         temp->data = i;
17         temp->next = listhead;
18         listhead = temp;
19     }
20
21     temp = listhead;
22     while (temp != NULL) {
23         printf("list item: current is %p; next is %p; data is %d\n", temp,
24             ↪ temp->next, temp->data);
25         temp = temp->next;
26     }
27     return 0;
28 }
```

- In the following code what it looks like in memory is like this:



- In this example the new element is added in the beginning of the list, you can add them at the end too, this is trivial for our purposes. Usually it is more efficient to add new elements to a start of a list.

3.4 To free or not to free?

- the free function needs to be called everytime we use malloc. But if the allocated memory is used through out all of the lifetime of the program we don't need to do that, that is if the memory remains referenced by pointers (through out all of the lifetime of the program) and remain in use the operating system will deallocate it after. (*This is actually not mentioned here but what happens is the OS will whipe the code page of the heap program in memory, thus we don't need to free it, but if we were to no longer need the memory we need to free it.)

3.5 Doubly linked lists

- In a singly linked list each element holds the address of the next element, but we can only move from the beginning to the right and not back, sometimes it is useful to be able to move to the left too. A doubly linked list is a list which holds an address to the next element and to the previous element.



3.6 Programming a doubly-linked List

- See the following code to get the doubly linked list.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 typedef struct listitem {
5     struct listitem *next;
6     struct listitem *prev;
7     int data;
8 } LISTITEM;
9 int main() {
10     LISTITEM head, *temp;
11
12     head.next = (LISTITEM*)&head;
13     head.prev = (LISTITEM*)&head;
14     head.data = -1;
15
16     for (int i = 0; i < 3; i++) {
17         temp = malloc(sizeof(LISTITEM));
18         temp->data = i;
19         temp->next = head.next;
20         head.next = temp;
21         temp->prev = &head;
22         temp->next->prev = temp;
23     }
24     return 0;
25 }
```

3.7 Initializing a doubly-linked list

- Initializing a doubly linked list is not as straight forward as initializing. With a doubly linked list the pointers are both ways. When adding the first element the first element by definition does not have a

previous element nor a next. We will add NULL pointers and when the next is added we will change.

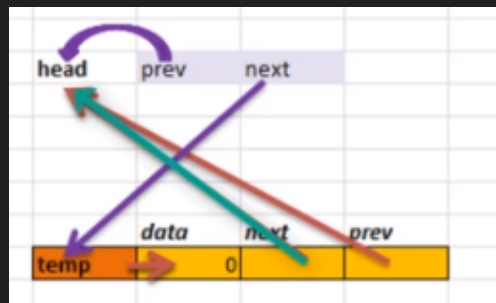
3.8 Implementing a doubly linked list

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 typedef struct listitem {
6     struct listitem *next; // pointer to next item.
7     struct listitem *prev; // pointer to prev item.
8     int data; // the data that this node will hold.
9 } LISTITEM;
10
11 int main() {
12     /* We need a head and a temporary node, the head will be the node in which the dll
13     ↪ will start. The temp will be the pointer to the nodes when we need to traverse
14     ↪ the dll. */
15     LISTITEM head, *temp;
16
17     /* Since the head element is the first node, we point the next and prev to itself,
18     ↪ since there are no other nodes to point at. It is also common for the first
19     ↪ node's next and prev to point to null, in this case we make it point to the
20     ↪ head. We initialize the data to be some arbitrary number, in this case -1. */
21     head.next = (LISTITEM*)&head;
22     head.prev = (LISTITEM*)&head;
23     head.data = -1;
24
25     /* We create a loop that will iterate from 0-2 (3 times).*/
26     for (int i = 0; i < 3; i++) {
27         /* Allocating the memory for this node. */
28         temp = malloc(sizeof(LISTITEM));
29         /* Given the memory already allocated we initialize the 'data' member of the
30         ↪ node. */
31         temp->data = i;
32         /* We get the current node's next pointer and point it to head.next, which in
33         ↪ this case will make a circular linked list. */
34         temp->next = head.next;
35         /* Next we get the head node and modify the 'next' pointer to this current
36         ↪ node, since now the head must point to the next node. */
37         head.next = temp;
38         /* We get the current node's prev pointer and point it to head.*/
39         temp->prev = &head;
40         /* Now that the next element is pointing to a node, we access the current
41         ↪ node's next pointer, then get the current node's next pointer's prev
42         ↪ pointer and point it to temp. Tricky. */
43         temp->next->prev = temp;
44     }
45
46     // Traversing a list forward.
47     temp = head.next; //
48     ↪ initialize our temporary variable to the head of the list
```

```

38 while (temp != &head) { //
    ↪ keep going until we've reach the end
39     printf("forward list item: current is %p; next is %p; prev is %p; data is
    ↪ %d\n", temp, temp->next, temp->prev, temp->data);
40     temp = temp->next; //
    ↪ move to the next item in the list
41 }
42
43 // Traversing a list backward.
44 temp = head.prev; //
    ↪ initialize our temporary variable to the tail of the list
45 while (temp != &head) { //
    ↪ keep going until we've reach the end
46     printf("backward list item: current is %p; next is %p; prev is %p; data is
    ↪ %d\n", temp, temp->next, temp->prev, temp->data);
47     temp = temp->prev; //
    ↪ move to the previous item in the
    ↪ list
48 }
49 return 0;
50 }

```



3.9 What is a queue?

- A queue is like a list in which the first element added is the first element served, FIFO First In First Out. As the elements are added, the order in which they are added are the order in which they are processed.
- Its common in operating systems, when there is a process to run or execute it gets added to the process queue and processed in the order in which it was added.
- A queue ensures that the correct order is maintained.

3.10 Queues

- The followign code is the adapted code from our doubly linked list turned into a queue.
- The enqueue function adds an element to the list and the dequeue removes it. The idea is that every element inserted usign enqueue, is dequeued in the order in which it was dequeued. Example enqueue(1), enqueue(2), enqueue(3) -i dequeue()=1, dequeue()=2 dequeue()=3.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // this what is going to be in the queue
6 typedef struct listitem {
7     struct listitem *next; // pointer to next item
8     struct listitem *prev; // pointer to previous item
9     int data;               // some data
10 } LISTITEM;
11
12 // this is the queue 'header'
13 typedef struct {
14     struct listitem *first; // pointer to first item
15     struct listitem *last;  // pointer to last item
16 } LISTHDR;
17
18 LISTHDR head; // declare this statically so that 'enqueue' and 'dequeue' can use it
19
20 // this puts an item in at the end of the queue
21 void enqueue(LISTITEM *item) {
22     LISTITEM *temp;
23
24     temp = head.last; // get the 'last' item in the
25     ↪ queue and keep hold of it
26     head.last = item; // put the item in the queue at
27     ↪ the end
28     item->prev = temp; // link back to old 'last' item
29     item->next = (LISTITEM*)&head; // set the forward link of the new item
30     temp->next = item; // and finally set the forward
31     ↪ link of the old 'last' item to the new one
32 }
33
34 // this removes an item from the front of the queue - returns the item or NULL if there
35 ↪ are no more items
36 LISTITEM *dequeue() {
37     LISTITEM *temp;
38
39     temp = head.first; // get the 'first' item
40     if (temp == (LISTITEM*)&head) { // if the head of the queue points to itself
41         ↪ ...
42         temp = NULL; // ... then the queue is empty
43     } else {
44         head.first = temp->next; // and set the queue header to point to the 'second'
45         ↪ item
46         head.first->prev = (LISTITEM*)&head;
47     }
48     return temp;
49 }
50
51 int main() {
52     LISTITEM *temp;
53
54 }

```

```

48 // first, make an empty queue
49 // ... which is a queue where the header points to itself and there are no items in
   ↳ it
50 head.first = (LISTITEM*)&head;
51 head.last = (LISTITEM*)&head;
52
53 for (int i = 0; i < 3; i++) {           // as before, populate the queue
54     temp = malloc(sizeof(LISTITEM)); // allocate some memory for the new queue item
55     temp->data = i;                     // set the item's data to the
   ↳ loop count so that we can see where it is in the queue
56     enqueue(temp);                     // and put it in the queue
57 }
58
59 printf("first item = %d\n", head.first->data);
60 printf("last item = %d\n\n", head.last->data);
61
62 // now let's dequeue from the end of the queue
63 do {                                     // keep going until the
   ↳ queue is empty
64     temp = dequeue();                   // if the queue is empty we will get
   ↳ NULL returned
65     if (temp != NULL) {
66         printf("data is %d\n", temp->data);
67         free(temp);                     // call 'free' to tidy up
68     }
69 } while (temp != NULL);
70
71 return 0;
72 }
73 /* output:
74 first item = 0
75 last item = 2
76
77 data is 0
78 data is 1
79 data is 2
80 */

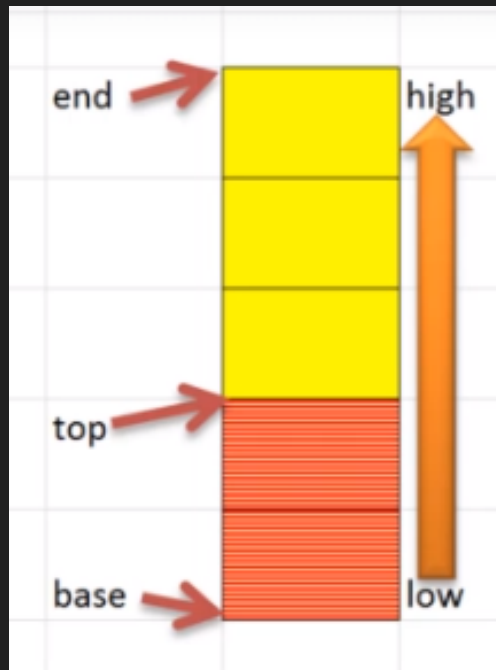
```

3.11 What is a stack?

- A stack is a data structure in which the last element that is added is the first element that comes out. An example of this is a stack of any object, you stack these objects one over another, when you remove one you simply get the last you added in.

3.12 Stacks

- A stack is usually implemented using a fixed size array. It is not usually implemented using linked lists of any kind. Thus it is faster because there are no pointers being accessed, it's simpler and much less memory is needed.



- A stack grows upward or downward, it all depends on how you push and pop, if you push an element if you make your stack grow downward, then when you pop it you must make sure that the one that you pushed last is the one that is popped.

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  typedef struct DATA {
5      int a; double b;
6  } DATA;
7  DATA *base, *end, *top;
8  #define STACKSIZE 5
9  int push(DATA *data) {
10     if (top == end) {
11         return 0;
12     } else {
13         memcpy(top, data, sizeof(DATA));
14         top++;
15         return 1;
16     }
17 }
18 DATA *pop() {
19     if (top == base) {
20         return NULL;
21     } else {
22         top--;
23         return top;
24     }
25 }
26 int main(int, char**) {
27     DATA data, *pdata;
28

```

```

29     base = (DATA*)malloc(STACKSIZE * sizeof(DATA));
30     end = base + STACKSIZE;
31     top = base;
32
33     // add the data items to the stack until stack overflow.
34     for (int i = 0; ; i++) {
35         data.a = i;
36         data.b = i * 2;
37         if (!push(&data)) {
38             break;
39         }
40     }
41
42     printf("First item: %d %lf\n", base->a, base->b);
43     pdata = base + 4;
44     printf("Last item: %d %lf\n\n", end->a, end->b);
45
46     do {
47         pdata = pop();
48         if (pdata != NULL) {
49             printf("data popped: %d %lf\n", pdata->a, pdata->b);
50         }
51     } while (pdata != NULL);
52
53     free(base);
54
55     return 0;
56 }
57 /* output:
58 First item: 0 0.000000
59 Last item: 0 0.000000
60
61 data popped: 4 8.000000
62 data popped: 3 6.000000
63 data popped: 2 4.000000
64 data popped: 1 2.000000
65 data popped: 0 0.000000
66 */

```

3.13 Calculating the length of a list

- Calculating the length of a linked list is a matter of traversing through all of the linked list counting. There is a much better way of doing this which is to simply keep track of each insertion and deletion, but for the purposes of this lets calculate it in this inefficient way.

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 typedef struct node {
4     struct node *next, *prev;
5     int data;
6 } node;
7 typedef struct list {
8     node *head, *tail;

```

```

9 } list;
10 unsigned int calculate_length(list *l) {
11     unsigned int length = 0;
12     node *current = l->head;
13     while (current != NULL) {
14         length++;
15         current = current->next;
16     }
17     return length;
18 }
19 void push(list *l, int data) {
20     node *new_node = (node*)malloc(sizeof(node));
21     new_node->data = data;
22     new_node->next = new_node->prev = NULL;
23     if ((l->head == NULL) && (l->tail == NULL)) {
24         l->head = l->tail = new_node;
25     } else {
26         l->tail->next = new_node;
27         new_node->prev = l->tail;
28         l->tail = new_node;
29     }
30 }
31 int pop(list *l) {
32     if ((l->head == NULL) && (l->tail == NULL)) {
33         return -999;
34     } else if (l->head == l->tail) {
35         int last = l->head->data;
36         free(l->head);
37         l->head = l->tail = NULL;
38         return last;
39     } else {
40         int last = l->tail->data;
41         l->tail = l->tail->prev;
42         free(l->tail->next);
43         l->tail->next = NULL;
44         return last;
45     }
46 }
47 int main(void) {
48     list l;
49     l.head = l.tail = NULL;
50     for (int i = 0; i < 10; i++) {
51         push(&l, i*2);
52         printf("pushed: %d\n", i*2);
53     }
54     printf("\n\nlength: %u\n\n", calculate_length(&l));
55     for (int i = 0; i < 11; i++) {
56         printf("popped: %d\n", pop(&l));
57     }
58     return 0;
59 }
60 /* output:
61 pushed: 0
62 pushed: 2

```

```
63 pushed: 4
64 pushed: 6
65 pushed: 8
66 pushed: 10
67 pushed: 12
68 pushed: 14
69 pushed: 16
70 pushed: 18
71
72
73 length: 10
74
75 popped: 18
76 popped: 16
77 popped: 14
78 popped: 12
79 popped: 10
80 popped: 8
81 popped: 6
82 popped: 4
83 popped: 2
84 popped: 0
85 popped: -999
86 */
```